56. Teknik İletişim: Ürün Yöneticileri ve Tasarımcılarla Konuşma Sanatı

'Jargon' Duvarını Yıkıp İş Değeriyle Konuşan Mühendis Olmak
Bölüm 10 • Zorluk Derecesi: Orta / İleri Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Yazılımcılar neden 'Bu kod çok çirkin, refactor etmemiz lazım' dediğinde patron bütçe vermez de, 'Bu refactor ödeme dönüşüm oranını %12 artıracak' dediğinde anında onaylar?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 Şirketler kod için değil; müşterilere değer üretmek ve ciro kazanmak için vardır. - 📌 Ürün Yöneticisi (PM) ve Tasarımcılarla konuşurken teknik jargonu (ORM, Big-O, Pod) bırakıp iş etkisine (Hız, Maliyet, Kullanıcı Kaybı) odaklanın. - 📌 Bir şeye 'Olmaz' demek yerine her zaman 'Şu maliyetle ve şu takasla (trade-off) yapılabilir' alternatifini sunun.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir oto tamircisine gittiğinizi düşünün.

Tamirci kaputu açıyor ve size şöyle diyor: *"Efendim krank milinin eksantrik segmanındaki mikrometre toleransı 0.04 bar sürtünme oluşturuyor, subapları rektifiye etmemiz lazım!"* Ne anlarsınız? Hiçbir şey! Korkarsınız ve tamirciye güvenmezsiniz.

Peki tamirci şöyle derse: *"Aracınızın motorundaki şu parça aşınmış. Eğer şimdi 500 TL'ye değiştirmezsek, haftaya uzun yolda motor yanar ve 50.000 TL masraf çıkar."* Anında onaylarsınız!

Yazılımda Ürün Yöneticileri (Product Manager) ve CEO'lar tam olarak o araba sahibidir. Onlara: *"PostgreSQL connection pool tükendi"* derseniz anlamazlar. *"Sunucu yoğunlukta siparişleri kaçırıyor ve günde 10.000 TL kaybediyoruz"* derseniz bütçeyi hemen alırsınız.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Jargondan İş Değerine Tercüme Sözlüğü:

```
TEKNİK JARGON (Yazılımcı Dili)          ——> İŞ DEĞERİ (Yönetici Dili)
_____________________________________________________________________

"Teknik borcumuz çok birikti."          ——> "Geliştirme hızımız yarıya düştü;
                                             yeni özellikleri 2 kat yavaş
çıkarıyoruz."

"Database index'i eklememiz lazım."     ——> "Arama sayfası 6 saniyeden 0.5 saniyeye
                                             inecek ve sepeti terk edenler azalacak."

"Test yazmamız gerekiyor."              ——> "Canlıya güncelleme çıkarken sistemin
                                             çökmesini ve müşteri kaybını
önleyeceğiz."

"AWS sunucu boyutunu büyütelim."        ——> "Aylık 20$ maliyetle cuma yoğunluğundaki
                                             kesintileri sıfıra indireceğiz."
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Teknik Ürün Yöneticisi ve İletişim Koçu
Ben bir backend mühendisiyim. Ürün Yöneticimize (Product Manager) 2 haftalık bir
'Teknik Borç ve Performans İyileştirme' sprinti için bütçe/zaman vermesi gerektiğini
anlatmak istiyorum.

## GÖREV:
1. Mühendislerin yaptığı en yaygın 3 iletişim hatasını listele.
2. Teknik borcu 'Para, Hız ve Müşteri Memnuniyeti' metrikleriyle anlatan ikna edici
1 sayfalık bir toplantı brifi hazırla.
3. PM'in 'Şu an yeni özellik çıkarmamız lazım, bunu erteleyelim' itirazına verilecek
profesyonel ve yapıcı karşı argümanı yaz.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ Acemi / Kontrolsüz AI Çıktısı (Riskli):

```
# Kötü Mühendis İletişimi:
"Kodlar spagetti oldu, kimse anlamıyor. Bu sprint yeni özellik yazamam, refactor
yapmam lazım."
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
# İş Odaklı Mühendislik Teklifi:

## 🎯 Konu: Ödeme Altyapısı Hızlandırma ve Çökme Önleme Yatırımı

Sayın Ürün Ekibi,

Son 3 sprint verilerimizi incelediğimizde:
- Ödeme sayfamız ortalama **4.2 saniyede** yükleniyor (Sektör ideali: < 1.5 saniye).
- Yavaşlık sebebiyle sepetten dönen kullanıcı oranımız yaklaşık **%8**.
- Ayrıca ödeme kodunun karmaşıklığı nedeniyle yeni bir ödeme yöntemi eklemek 2 hafta sürüyor.

### 💡 2 Haftalık Odaklanmış İyileştirme Planımız:
1. Veritabanı sorgularını optimize ederek sayfa yüklenme süresini **1.2 saniyeye** indireceğiz.
2. Ödeme kodunu modüler hale getirerek bir sonraki çeyrekte planlanan 'Apple Pay' entegrasyon süresini **2 haftadan 3 güne** düşüreceğiz.

### 📈 Beklenen İş Getirisi:
- Sepet tamamlama oranında tahmini **%5-%7 ciro artışı**.
- Gelecek sprintlerde yeni özellik teslim hızında **%40 ivmelenme**.
```

💡 **Kod Analizi:** Bu dille konuşan bir mühendis, şirketin büyümesini dert edinen bir iş ortağıdır. PM bu teklifi reddetmez; tam aksine heyecanla destekler.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Toplantılarda teknik jargona boğulup PM'in veya tasarımcının gözünü korkutmak.
  - **Çözüm:** Metaforlar, basit analojiler ve kullanıcı deneyimi metrikleriyle konuşun.
- **Tuzak:** Tasarımcının çizdiği zor bir UI için 'Bu imkansız, yapılmaz' demek.
  - **Çözüm:** 'Bunu birebir yapmak 3 hafta sürer; ancak şu küçük değişiklikle 2 günde %90 benzerini yapabiliriz' deyin.
- **Tuzak:** Şirketin para kazanma modelini (Business Model) hiç merak etmemek.
  - **Çözüm:** Şirketinizin nereden para kazandığını ve hangi özelliğin ciro getirdiğini mutlaka öğrenin.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 56: Teknik İletişim - Mühendislikten İş Değerine Köprü*

- 🖥️ **Slayt 1:** Neden şirketler kod için değil, değer için yaşar?

- 🖥️ **Slayt 2:** Jargon duvarını yıkmak: Teknik dilden iş diline tercüme.
- 🖥️ **Slayt 3:** Tasarımcılar ve PM'lerle 'Yapıcı Takas (Trade-off)' müzakeresi.
- 🖥️ **Slayt 4:** Teknik borcu yönetime satma ve onay alma stratejisi.

Canlı Kodlama / Ekran Kaydı Senaryosu:
Bir sprint planlama toplantısı canlandırın. 'Kodu yeniden yazmalıyız' diyen mühendis ile 'Bu optimizasyon müşteri dönüşümünü %8 artıracak' diyen mühendisin toplantıdaki etkisini karşılaştırın.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Şirketinizdeki veya projenizdeki bir teknik ihtiyacı alın. Bunu tek bir teknik terim kullanmadan, bir işletme sahibine anlatır gibi 3 cümlelik bir iş teklifine dönüştürün.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Bir yazılım mühendisi ürün yöneticisine (PM) teknik bir borcu açıklarken hangi yaklaşımı seçmelidir? - A) 'Kodlar çok kötü, sen anlamazsın' demek - B) Teknik borcun geliştirme hızını nasıl yavaşlattığını, yeni özelliklerin çıkış süresini ne kadar geciktirdiğini ve çözüldüğünde şirkete sağlayacağı hız/ciro getirisini anlatmak - C) Sadece kızgın emojiler atmak - D) Hiçbir şey söylemeden gizlice kodu değiştirmek > **Doğru Yanıt:** B > **Açıklama:** Yöneticiler iş sonuçlarıyla ilgilenir. Teknik borcu geliştirme hızı, hata riski ve finansal getiri üzerinden açıklamak anında onay almanızı sağlar.

Soru 2: Bir tasarımcının sunduğu arayüzü kodlamak çok zorsa profesyonel bir mühendis nasıl yanıt vermelidir? - A) 'Bu imkansız, çizimi çöpe at' demek - B) Tasarımın neden zor olduğunu açıklayıp, aynı kullanıcı hedefine hizmet eden ve çok daha kısa sürede yapılabilecek pratik bir alternatif önermek - C) Tasarımcıyla küsmek - D) Çizimi görmezden gelip kafasına göre yapmak > **Doğru Yanıt:** B > **Açıklama:** Mühendislik takasları (trade-off) yönetme sanatıdır. 'Olmaz' demek yerine alternatif çözüm sunmak saygı uyandırır.